lab3 实验报告

PB24111621 肖瀚

一、实验目的与内容

- 实验目的：根据数据通路，搭建一个简单单周期 CPU。
- 实验内容：通过 PC 寄存器、寄存器堆、译码器、数据选择器、ALU 等模块的数据通路连接，设计简单单周期 CPU，并在仿真框架下进行仿真并查看波形图，然后利用 PDU 进行上板测试，并回答思考题。

二、逻辑设计

译码器设计

本次实验要求实现的指令，可分为三类指令：R-type, I-type, U-type，以下是译码器的实现思路：

1. 具体指令的确定：首先提取 opcode(inst[6:0]), rf_wa(inst[11:7]), funct3(inst[14:12]),rf_ra0(inst[19:15]), rf_ra1(inst[24:20]), funct7(inst[31:25]) ；然后先根据 opcode 对指令进行初步分类(lui, auipc此时已可指定，分别利用 SRC1, ADD指令实现)，再利用 funct3, funct7 确定其他的指令。
2. 立即数确定：对 I-type，立即数为 inst[31:20] 进行符号扩展；对 U-type，立即数为 inst[31:12] 左移 12 位，对 R-type 以及其他，立即数为 0。
3. 选择信号的确定：对 alu_src0_sel，0 = rf_ra0，1 = pc；对 alu_src1_sel，0 = rf_ra1，1 = imm。对 R-type，二者组合为 00；对 I-type，二者组合为 01；对 lui，二者组合为 01；对 auipc，二者组合为 11。

CPU 搭建

根据数据通路，例化 PC 寄存器，从指令存储器读取相应的指令，在 global_en 的控制下，每个周期 npc 会自动加 4，实现指令的逐条读取；读取指令后送入译码器，产生指令码、三个寄存器的地址、写使能信号、数据选择信号、立即数；根据寄存器地址，在寄存器堆读取相应的数据，经过数据选择器的选择后送入 ALU(寄存器数据、PC、imm)，经过 ALU 运算后生成结果，再将结果写入寄存器堆，完成一条指令。

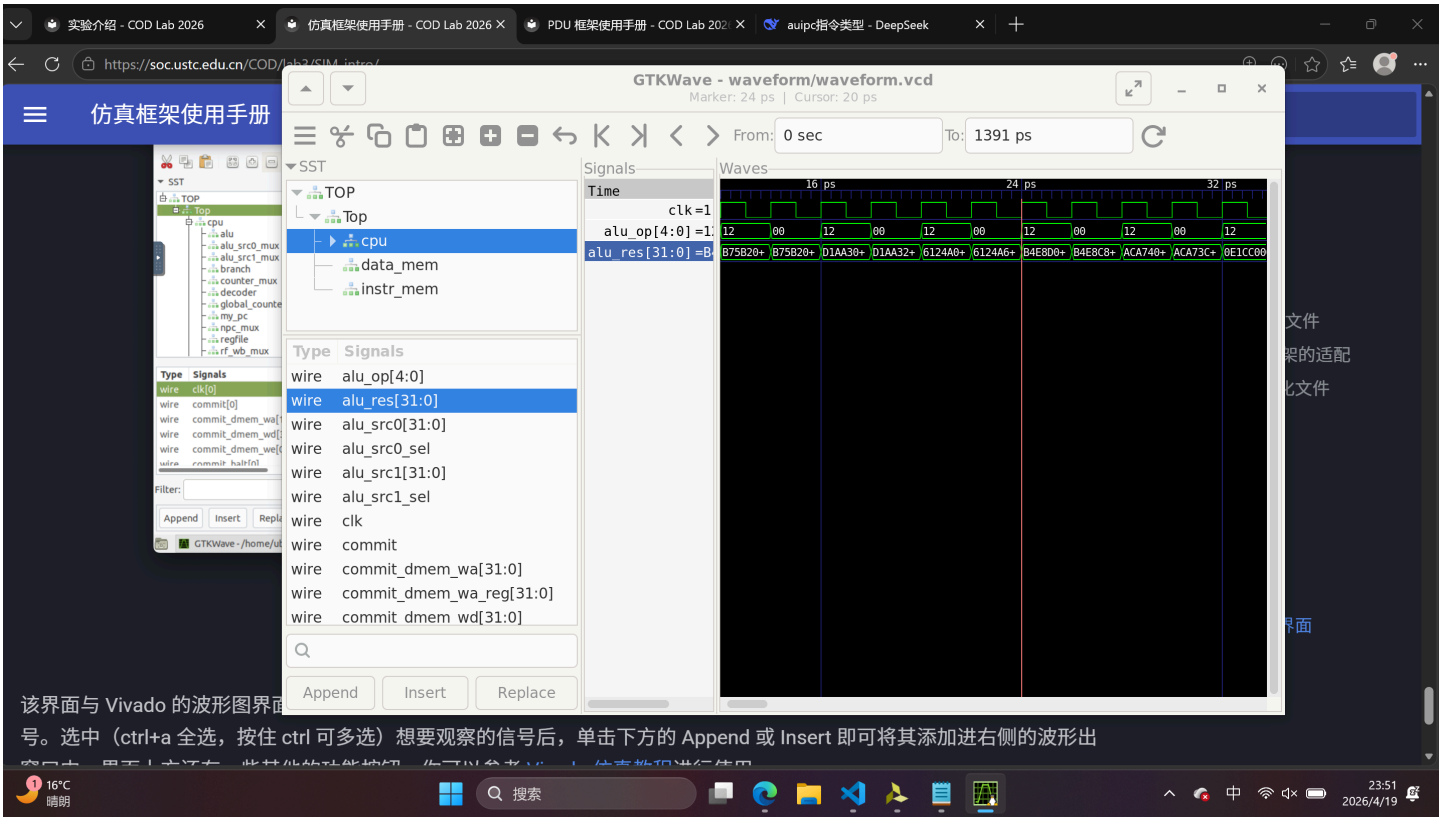
三、结果仿真与测试

对于**仿真框架**的仿真，经过一系列尝试，最终终端输出结果如下：

```
qi@ISLA:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Hit good trap.
Simulation finished.
Time:                           1392
Instruction count:              695
PC:                             0x00400AD8
Instruction:                    0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0xFFFFFDB2 [ 2] = 0xB5D38AC0 [ 3] = 0x32FAEA78
[ 4] = 0xFFFFFFEF [ 5] = 0x00000635 [ 6] = 0xA7A77000 [ 7] = 0xE485EA6C
[ 8] = 0x05DB1AB4 [ 9] = 0x0000002F [10] = 0xE01B3AA8 [11] = 0x26B6CACC
[12] = 0xFFFFFE3F [13] = 0xEFE96000 [14] = 0xC5A3FA9C [15] = 0x63B04A90
[16] = 0x96E44A60 [17] = 0x00000030 [18] = 0x00000000 [19] = 0xB5D39144
[20] = 0x0A356000 [21] = 0x736B5000 [22] = 0xD8456000 [23] = 0xD2FEF000
[24] = 0x63B050F4 [25] = 0xC5A40108 [26] = 0x05DB2130 [27] = 0xA68D1000
[28] = 0x00000030 [29] = 0xA7A77000 [30] = 0xA922DA84 [31] = 0x26B6D158
qi@ISLA:~/COD25-Simulation-Framework$
```

这与助教所给的数据信息是一致的

简单查看波形图如下：



对于上板的测试，利用 PDU 仿真框架，建好两个 ip 核，用对应的 coe 文件初始化，在项目中导入 vsrc 文件夹，最后烧写比特流文件，在 FPGA 上运行测试，最终结果展示如下：

Program success!

FPGA interface

led7 led6 led5 led4 led3 led2 led1 led0

A14 A13 A16 A15 B13 C13 B16 B15

FPGA
XC7A35t-FGG484-1

D19 E19 C19 C18 C17 D17 B18 B17

sw7 sw6 sw5 sw4 sw3 sw2 sw1 sw0

button
(F20)

Mouse Movements

X: 0

Y: 0

```
User: RR;\nR;RR 0 32;
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000
00000000 00000000 00000000 00000000

User: R[illegible]AU----------

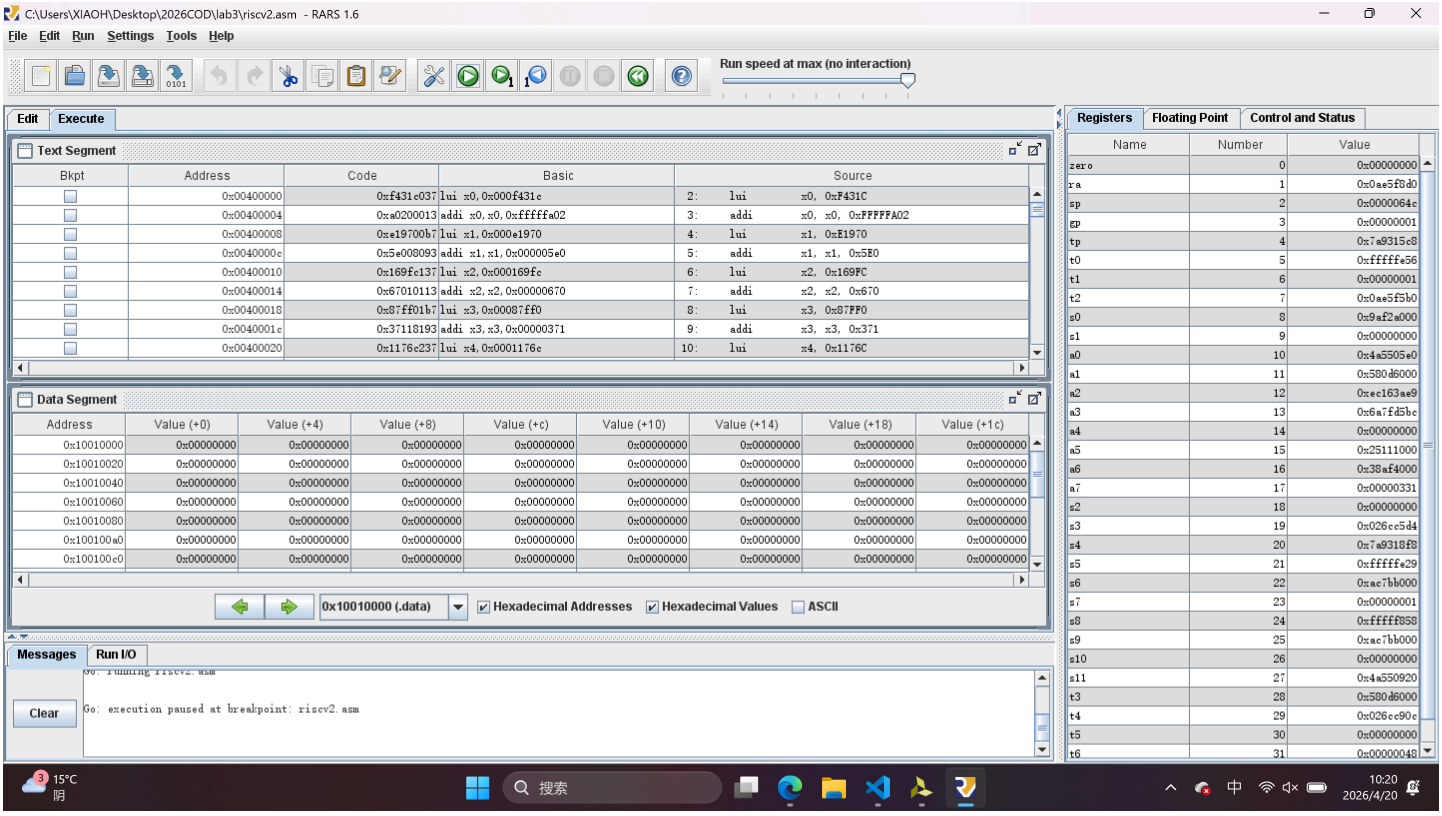
User: RR 0 32;
00000000 0AE5F8D0 0000064C 00000001
7A9315C8 FFFFFE56 00000001 0AE5F5B0
9AF2A000 00000000 4A5505E0 580D6000
EC163AE9 6A7FD5BC 00000000 25111000
38AF4000 00000331 00000000 026CC5D4
7A9318F8 FFFFFE29 AC7BB000 00000001
FFFFF858 AC7BB000 00000000 4A550920
580D6000 026CC90C 00000000 00000048
```

uart pins: rxd txd
xdc,ucf sym: F15 F13
baud rate: 9600

RR 0 32;

input

与 rars 的运行结果(如下图)比对，可知是一致的。



四、思考与总结

1. **Q：本次实验的 CPU 中，哪些模块用到了时钟信号？**
   A：PC, regfile, 以及 CPU 模块的 debug 部分也使用了时钟信号。
2. **Q：请分别给出一条指令，以符合下面的描述：**
   - alu_src0 选择 pc： `auipc x1, 0x1000`
   - alu_src0 选择 rf_rd0： `add x1, x2, x3`
   - alu_src1 选择 rf_rd1： `add x1, x2, x3`
   - alu_src1 选择 imm： `addi x1, x2, 66`
3. **Q：请指出本次实验的 CPU 中可能的关键路径；如果这条路径的延迟大于一个时钟周期，可能会带来什么影响？**

关键路径如下：

```
PC → IM → 译码器 → 寄存器堆 → 数据选择器 → ALU → 寄存器堆
```

如果这条路径的延迟大于一个时钟周期，将导致建立时间违例：在时钟上升沿到来时，后续寄存器（如寄存器堆或 PC）无法捕获到稳定正确的数据，从而写入错误的值，程序执行结果不可预测，整个 CPU 将无法正常工作。

总结： 本次实验中，实现了指令译码器，并根据数据通路实现了简单单周期 CPU，并配置好了仿真框架的相关环境，进行仿真测试，为后续实验打下基础。